

Aula de JavaScript: Criando um sistema de Busca de CEPs

Nesta aula, vamos aprender como criar passo a passo uma aplicação simples de busca de CEPs usando JavaScript, HTML e CSS. O objetivo é que o aluno entenda cada etapa e possa implementar sozinho.

Passo 0 — Estrutura de pastas

Primeiro, organize o projeto assim:

```
cep/  
├─ index.html  
├─ style.css  
└─ script.js
```

O `index.html` terá a estrutura da página, `style.css` será o estilo e `script.js` conterá toda a lógica em JavaScript.

1 Passo 1 — Criar o HTML básico

No `index.html`, crie a estrutura básica:

```
<!DOCTYPE html>  
<html lang="pt-br">  
<head>  
  <meta charset="UTF-8">  
  <title>Busca de CEPs</title>  
  <link rel="stylesheet" href="style.css">  
</head>  
<body>  
  
  <h1>Busca de CEPs</h1>  
  
  <div class="busca-container">  
    <input type="text" id="cep-input" placeholder="Digite o CEP">  
    <button id="btn-buscar">Buscar CEP</button>  
  </div>
```

```
<div class="cep-container" id="cep-container"></div>

<script src="script.js"></script>
</body>
</html>
```

Aqui criamos:

- Um `input` para o usuário digitar o CEP;
- Um `button` que vai disparar a busca;
- Um container onde os resultados serão exibidos.

2 Passo 2 — Estilizando a página

No `style.css`, adicione estilo simples:

```
body {
  font-family: "Segoe UI", sans-serif;
  background: #f9f9f9;
  padding: 20px 40px;
}
.busca-container {
  margin-bottom: 20px;
}
input {
  padding: 8px;
  font-size: 16px;
}
button {
  padding: 8px 15px;
  background: #007acc;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}
.cep-container {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
}
.cep-card {
  background: #eef6fb;
  padding: 15px;
  border-radius: 6px;
```

```
flex: 1 1 250px;  
}
```

Isso garante que os inputs, botão e cards fiquem visualmente organizados.

3 Passo 3 — Criando a função de busca no JavaScript

No `script.js`, vamos criar uma função assíncrona que consulta a API ViaCEP:

```
async function buscarCep(cep) {  
  try {  
    const res = await fetch(`https://viacep.com.br/ws/${cep}/json/`);  
    if(!res.ok) throw new Error("Erro na requisição");  
    const dados = await res.json();  
    if(dados.erro) throw new Error("CEP não encontrado");  
    return dados;  
  } catch(err) {  
    alert("Erro: " + err.message);  
    return null;  
  }  
}
```

O que acontece aqui:

- Usamos `fetch` para fazer a requisição HTTP;
- `await` espera a resposta;
- Tratamos erros com `try/catch`;
- Se o CEP não existir, mostramos um alerta.

4 Passo 4 — Criando os cards de resultado

Criamos uma função que recebe os dados do CEP e gera um card na tela:

```
function adicionarCard(dados) {  
  const container = document.getElementById("cep-container");  
  const div = document.createElement("div");  
  div.className = "cep-card";  
  div.innerHTML = `  
    <h3>CEP: ${dados.cep}</h3>  
    <p>Logradouro: ${dados.logradouro || "-"}</p>  
    <p>Bairro: ${dados.bairro || "-"}</p>  
    <p>Cidade: ${dados.localidade}</p>  
  `
```

```
<p>UF: ${dados.uf}</p>
`
container.appendChild(div);
}
```

Explicação:

- Criamos um elemento `div` e adicionamos classes para estilo;
- Usamos template strings para preencher os dados;
- Adicionamos a `div` ao container principal.

5 Passo 5 — Ligar o botão à função

Adicione o evento de clique para buscar o CEP e gerar o card:

```
document.getElementById("btn-buscar").addEventListener("click", async () => {
  const input = document.getElementById("cep-input");
  const cep = input.value.trim();
  if(cep.length === 0){
    alert("Digite um CEP válido");
    return;
  }
  const dados = await buscarCep(cep);
  if(dados) adicionarCard(dados);
  input.value = "";
  input.focus();
});
```

Passo a passo:

- Pegamos o valor do input e removemos espaços;
- Validamos se não está vazio;
- Chamamos a função assíncrona `buscarCep` ;
- Se houver resultado, chamamos `adicionarCard` ;
- Limpa e foca o input para o próximo CEP.

6 Passo 6 — Testando a aplicação

- Abra o `index.html` no navegador;
- Digite um CEP válido, como `28020030` ;
- Clique em "Buscar CEP" e veja o card aparecer;

- Repita para adicionar múltiplos CEPs.



Dicas finais

- Pode criar melhorias: máscara de CEP, spinner de carregamento, validação de números;
- Esse exercício ensina: manipulação do DOM, fetch API, async/await e tratamento de erros;
- Você pode dividir em funções menores para organizar melhor.